

SmartFight — Team Split & Task Board

Date: 2026-04-24

Team size: 2 developers (Pattern A — full-stack, similar levels)

Scope: Phases 1-5 task allocation

Effort key: S ~2h M ~4-6h L ~1 day+

Division Principles

Work is divided so each developer owns a **vertical feature track** end-to-end — from database entity through service, controller, and template. This produces topic experts rather than layer specialists, avoids merge conflicts through clean file ownership, and keeps both developers engaged across the full project lifecycle.

Dev A — Identity & Events Track Owns: <i>who fights, where, when</i> - Fighter entity and CRUD - Event entity and CRUD - WeightClass entity - Public events pages - Rankings (admin + public) - Registration / signup - VichUploader (photos, posters)	Dev B — Fights & Stats Track Owns: <i>the actual fighting</i> - MatchProposal entity and CRUD - FightResult entity and CRUD - FightStat entity - All four enum classes - Stats entry UI (grid + CSV) - Admin commands - Status automation
---	--

Shared responsibilities (both devs)

- The migration.** Dev A writes it, Dev B reviews every line. Both present when applied to a real database.
- Security config changes.** Four eyes on every `security.yaml` edit.
- Enum conventions.** Dev B writes the first enum, both agree the pattern, then Dev B finishes the rest.
- Code review.** Dev A reviews Dev B's PRs and vice versa. No self-merging. Minimum one substantive comment per PR.
- Daily 15-minute sync:** what merged yesterday, what's in progress today, any blockers.

Phase 1 — Foundation

Dependencies within this phase

- Dev A delivers `WeightClass` entity first — Dev B needs it for `MatchProposal.weightClass` FK
- Dev B delivers enum classes first — Dev A needs `EventStatus` for `Event` modifications
- Migration is written last, after all entities are ready

Dev A — Phase 1 Tasks

- **P1-A1** Create `WeightClass` entity + repository **S**
- **P1-A2** Modify `Fighter` entity (firstName, lastName, countryCode, discipline FK, nullable user FK, drop legacy weight class fields) **M**
- **P1-A3** Restructure `Event` entity (startsAt/endsAt, venue fields, posterUrl, drop legacy) **M**
- **P1-A4** Add `event_poster` mapping to `vich_uploader.yaml` **S**
- **P1-A5** Author the Phase 1 migration (collaborate with Dev B on review) **L**
- **P1-A6** Create `RegistrationController` **S**
- **P1-A7** Create `RegistrationFormType` with validators **M**
- **P1-A8** Create `templates/security/register.html.twig` **S**
- **P1-A9** Update `security.yaml` `access_control` for `/register` **S**
- **P1-A10** Add "Sign Up" link to front base template **S**

Dev B — Phase 1 Tasks

- **P1-B1** Create `EventStatus` enum class (first — establishes pattern) **S**
- **P1-B2** Create `MatchStatus`, `CardType`, `FightMethod` enum classes **S**
- **P1-B3** Modify `MatchProposal` entity (scheduledRounds, isTitleFight, weightClass FK, odds, cardPosition, cardType) **M**
- **P1-B4** Modify `FightResult` entity (knockdowns, method enum constraint) **S**
- **P1-B5** Create `FightStat` entity + repository **S**
- **P1-B6** Create `Ranking` entity + repository **S**
- **P1-B7** Create `SystemMeta` entity + repository **S**
- **P1-B8** Create `CreateAdminCommand` with interactive prompts and `--force` flag **M**
- **P1-B9** Create `EventStatusService` (shared logic) **M**
- **P1-B10** Create `RefreshEventStatusCommand` **S**
- **P1-B11** Create `StatusRefreshListener` (kernel.request subscriber) **M**
- **P1-B12** Migration review pass (review Dev A's migration SQL) **M**

Phase 1 acceptance gate: Before Phase 2 begins, the migration runs clean, `doctrine:schema:validate` passes, registration works end-to-end, and `app:create-admin` produces a working admin account.

Phase 2 — Admin CRUD

Scope

Full administrative interface for managing all new and modified entities. Each dev implements the admin controller, form type, templates, and tests for entities in their track.

Dev A — Phase 2 Tasks

- **P2-A1** Admin Fighter CRUD controller (list, new, edit, delete) **M**
- **P2-A2** Fighter form type with VichUploader photo field **M**
- **P2-A3** Fighter admin templates (list, form, detail) **M**
- **P2-A4** Country code select with flag preview **S**
- **P2-A5** Admin Event CRUD controller **M**
- **P2-A6** Event form type with poster upload, venue fields **M**
- **P2-A7** Event admin templates **M**
- **P2-A8** "Cancel Event" action (status → CANCELLED) **S**

Dev B — Phase 2 Tasks

- **P2-B1** Admin MatchProposal CRUD controller **M**
- **P2-B2** MatchProposal form type (fighters, weight class, odds, rounds, title flag, card position, card type) **L**
- **P2-B3** MatchProposal admin templates **M**
- **P2-B4** Admin FightResult CRUD controller **M**
- **P2-B5** FightResult form type (method enum, winner, round/time, knockdowns) **M**
- **P2-B6** FightResult admin templates **M**
- **P2-B7** Link FightResult creation to a specific MatchProposal (enforces integrity) **S**

Shared Phase 2 work

- Agree on a consistent admin form/template style (one pair-programming session at phase start)
- Agree on flash message conventions (success/error/warning)
- Validate no existing admin screens broke from Phase 1 migrations

Phase 3 — Public Events Browsing

Scope

Public-facing event pages inspired by ufc.com. Dev A owns this entire phase since it's squarely in the Events track.

Dev A — Phase 3 Tasks

- **P3-A1** `EventController::upcoming()` action + route `/events` S
- **P3-A2** `EventController::past()` action + route `/events/past` S
- **P3-A3** `EventController::detail()` action + route `/events/{id}` M
- **P3-A4** `EventRepository::findUpcoming()`, `findPast()` methods S
- **P3-A5** Upcoming events list template (poster, headline fight, date, venue, "How to Watch") M
- **P3-A6** Past events list template (result badge instead of watch button) M
- **P3-A7** Event detail template with MAIN_CARD / PRELIMS tabs L
- **P3-A8** Fight card row component (fighter photos, names, flags, odds) M
- **P3-A9** Poster fallback logic (use headline fighter photos if no poster uploaded) S

Dev B — Phase 3 Tasks (supporting)

- **P3-B1** Provide result summary partial for past events (called by Dev A's templates) S
- **P3-B2** Odds display partial (decimal format, with styling) S
- **P3-B3** Code review Dev A's event controllers and templates M
- **P3-B4** Begin Phase 4 prep: draft FightStat admin UX plan M

Load balancing note: Phase 3 is Dev A-heavy by design. Dev B uses the slack to start Phase 4 prep — since Phase 4 is the largest single workstream in the project, early preparation is valuable.

Phase 4 — Fight Statistics

Scope

The largest and most complex phase. Dev B owns this entire track. Dev A supports with reviews and minor components.

Dev A — Phase 4 Tasks (supporting)

- **P4-A1** Code review Dev B's stats entry UI **M**
- **P4-A2** Code review Dev B's CSV import/export logic **M**
- **P4-A3** Prep Phase 5: draft rankings admin UX plan **M**
- **P4-A4** Integrate stats display partial into event detail page (Phase 3 extension) **S**

Dev B — Phase 4 Tasks

- **P4-B1** `FightStatRepository` query methods (totals, per-round, by fight) **M**
- **P4-B2** `FightStatService` (CRUD, validation rules, totals vs per-round logic) **L**
- **P4-B3** Admin stats entry UI — quick totals form (4 fields) **M**
- **P4-B4** Admin stats entry UI — detailed per-round grid (keyboard-optimized, live totals) **L**
- **P4-B5** "Copy previous round" button logic **S**
- **P4-B6** Validation: landed $\leq$ thrown, rounds $\leq$ scheduled, knockdown sum consistency **M**
- **P4-B7** CSV export endpoint (per-fight download) **M**
- **P4-B8** CSV empty template generator **S**
- **P4-B9** CSV import endpoint with full validation **L**
- **P4-B10** Public stats display partial (totals + optional round tabs, image 5 layout) **M**
- **P4-B11** Accuracy % computation in views **S**

Phase 5 — Rankings

Scope

Manual rankings system with champion designation + top 10 per weight class. Dev A owns this track. Dev B supports.

Dev A — Phase 5 Tasks

- **P5-A1** `RankingRepository` query methods (by weight class, with fighter joins) **M**
- **P5-A2** `RankingService` (reorder, set champion, validate uniqueness) **M**
- **P5-A3** Admin rankings controller (list weight classes, edit one) **M**
- **P5-A4** Admin rankings edit UI (champion slot + drag-reorder top 10) **L**
- **P5-A5** "Vacant" state handling for champion slot **S**
- **P5-A6** Fighter autocomplete for adding rankings **M**
- **P5-A7** Public `/rankings` route and controller **S**
- **P5-A8** Public rankings template (4-column grid matching images 2 & 3) **M**
- **P5-A9** Champion card styling (photo, belt, "CHAMPION" label) **S**

Dev B — Phase 5 Tasks (supporting)

- **P5-B1** Code review Dev A's rankings admin UI **M**
- **P5-B2** Code review Dev A's public rankings page **S**
- **P5-B3** Cross-link: fighter detail page shows their ranking (if any) **S**
- **P5-B4** Final QA pass: test all Phase 1-5 features end-to-end **L**

Workflow Rules

Branching

Each dev works on short-lived personal branches off `main`. Max 2-3 days before merging. One pull request per ticket or small group of related tickets. No direct commits to `main`.

Daily sync (15 minutes)

Three questions: what merged yesterday, what's in progress today, any blockers. If one dev is waiting on the other's deliverable, surface it here.

Code review requirements

- Dev A reviews all Dev B pull requests; Dev B reviews all Dev A pull requests
- No self-merging under any circumstances
- Minimum one substantive comment per review — "LGTM" is not sufficient evidence of review
- Reviewer runs the code locally before approving (don't just read it)

Migration runs are joint

When the Phase 1 migration (or any later schema change) is applied to a shared database, both devs are on the call. Database dump taken together, migration run together, `doctrine:schema:validate` run together.

Pre-commit checklist

- `php -l` on changed files (syntax check)
- `php bin/console doctrine:schema:validate` (if entities touched)
- Manual smoke test of the feature just changed

Documentation ownership

Whoever owns a feature writes its README section. Dev A writes the fighter/event/ranking sections. Dev B writes the match/result/stats sections. Updates happen as features ship, not at the end.

Workload Summary

Phase	Dev A focus	Dev A tasks	Dev B focus	Dev B tasks
Phase 1	Entities, registration, migration	10	Enums, commands, automation	12
Phase 2	Fighter & Event admin	8	Match & Result admin	7
Phase 3	Public events pages	9	Support + Phase 4 prep	4
Phase 4	Support + Phase 5 prep	4	Full stats system	11
Phase 5	Full rankings system	9	Support + final QA	4
Total		40		38

Workload is balanced across the full project even though individual phases are weighted toward one dev. Phases 3 and 5 favor Dev A; Phase 4 favors Dev B. The quieter dev in any given phase uses their spare bandwidth to review PRs, prepare for the next phase, and keep documentation current.

Ownership Map (Quick Reference)

Entity / System	Owner
Fighter	Dev A
Event	Dev A
WeightClass	Dev A
Ranking	Dev A
Registration / signup	Dev A
Public events pages	Dev A
Public rankings page	Dev A
VichUploader configuration	Dev A
MatchProposal	Dev B
FightResult	Dev B
FightStat	Dev B
SystemMeta	Dev B
All enum classes	Dev B
CreateAdminCommand	Dev B
Status automation (command + listener + service)	Dev B
Stats entry UI and CSV import/export	Dev B
Phase 1 migration	Dev A writes, Dev B reviews
security.yaml	Four-eyes required